

From CSV to checkpoint

Every function and class the fine-tuning path needs, in the order it runs, with what it does and why it exists. Exploration cells are left out.

■ Driver side: the notebook and the Ray Data tasks it launches ■ Worker: runs inside each Ray Train worker

The whole pipeline

Top to bottom is execution order. Everything above `trainer.fit()` prepares data and settings in the notebook. The orange block is the one function Ray copies into every worker.



TorchTrainer(...).fit()

Frees the notebook's BERT copies first, then starts the workers, splits train/val into shards and runs the loop below in each

train_loop_per_worker(config)

× num_workers, all at the same time

set_seeds() + train.get_dataset_shard("train" / "val")

Fresh process, so seed again. Take only this worker's shard of each dataset

BertModel.from_pretrained → FinetunedLLM → prepare_model

SciBERT + new random head → move to this worker's device, wrap in DDP, copy worker 0's weights to all

CrossEntropyLoss · Adam · ReduceLROnPlateau

Built after `prepare_model` so the optimizer holds the device-side parameters. Batch per worker = `batch_size // world_size`

for epoch in range(num_epochs)

train_step()

One pass over the train shard, updating weights every batch

eval_step() → scheduler.step(val_loss)

One pass over the val shard with no updates; lower the learning rate if val loss has stalled

model.module.save(dp) → train.report(metrics, checkpoint)

Write `args.json` + `model.pt` to a temp dir; Ray copies it to `~/ray_results` and logs metrics. All workers wait for each other here

Workers stay in sync in three places: `prepare_model` (same starting weights), every `backward()` (gradients averaged) and every `train.report`.

results

`results.metrics_dataframe` (one row per epoch) · `results.best_checkpoints` (ranked by `val_loss`)

FinetunedLLM.load() → predict / predict_proba → decode

Next step, not in the notebook yet: rebuild the model from the best checkpoint and turn indices back into tag names

One training step, one batch

What happens to a single batch inside `train_step`. Shapes use your settings: `batch_size` 32 // 2 workers = 16 rows per worker, and `L` is the longest sequence in the batch (at most 128).



The step always runs in this order: `zero_grad` → `forward` → `loss` → `backward` → `step`. `eval_step` does only forward and loss, under `model.eval()` and `torch.inference_mode()`.

Every function, in the order it's needed

One line on what each does and one on why the pipeline needs it.

Setup

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>set_seeds(seed=42)</code>	Seeds numpy, random, torch and CUDA; makes cuDNN deterministic; sets <code>PYTHONHASHSEED</code> .	Reproducible runs. It's called again in every worker because each worker is a new process.	BOTH
<code>ray.init()</code>	Starts a local Ray cluster. <code>preserve_order=True</code> is set on the data context first.	Everything with <code>ray.data</code> and <code>ray.train</code> runs as Ray tasks and actors.	DRIVER

Data CSV → shuffled, stratified Ray Datasets

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>load_data(num_samples=None)</code>	<code>read_csv</code> with <code>override_num_blocks</code> = CPU count, seeded shuffle, optional <code>take(num_samples)</code> .	A single CSV would otherwise load as one block, which means no parallelism.	DRIVER
<code>stratify_split(ds, stratify, test_size, shuffle=True, seed=1234)</code>	<code>groupby(tag).map_groups(_add_split)</code> labels rows train/test within each tag; <code>_filter_split</code> separates them; both get shuffled. Inner helpers: <code>_add_split</code> , <code>_filter_split</code>	Ray's <code>train_test_split</code> has no <code>stratify</code> . Splitting each tag separately keeps class proportions without pulling the data onto one machine.	DRIVER

Preprocessing raw rows → token ids, masks, targets

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>clean_text(text)</code>	Lowercase, remove stopwords (the regex is compiled once and cached on the function), space out and strip punctuation, collapse spaces, drop links.	Less noise and fewer wasted tokens.	DRIVER

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>tokenize(batch)</code>	SciBERT tokenizer: pad to the longest in this batch, truncate at 128. Returns <code>ids</code> , <code>masks</code> , <code>targets</code> .	BERT takes token ids plus an attention mask, not text.	DRIVER
<code>preprocess(df, class_to_index)</code>	title + description → <code>text</code> , <code>clean_text</code> , keep text/tag, map tag → index, <code>tokenize</code> .	One function Ray can apply to every batch with <code>map_batches</code> .	DRIVER
CustomPreprocessor subclass of Ray's <code>Preprocessor</code>	<code>_fit</code> builds <code>class_to_index</code> / <code>index_to_class</code> from the train set. <code>_transform_pandas</code> calls <code>preprocess</code> . Use the public <code>fit</code> / <code>transform</code> / <code>fit_transform</code> .	Fit on train only, and val reuses the same mapping, so nothing leaks. It also keeps the mapping for decoding predictions later.	DRIVER
<code>ds.materialize()</code>	Executes the lazy pipeline once and keeps the blocks in Ray's object store.	Without it, the read, split and tokenize steps re-run every epoch.	DRIVER
<code>decode(indices, index_to_class)</code>	Class indices → tag names.	Turns model output back into labels a person can read.	DRIVER

Model SciBERT + classification head

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
FinetunedLLM (<code>llm</code> , <code>embedding_dim</code> , <code>num_classes</code> , <code>dropout_p</code>)	<code>forward: ids, masks</code> → BERT pooled output (768) → Dropout → <code>fc1</code> → logits.	Reuses a pretrained encoder and adds a small head for your tags.	WORKER
<code>.predict(batch) / .predict_proba(batch)</code>	In <code>eval</code> and <code>inference_mode</code> : argmax of the logits / softmax probabilities.	Inference. Softmax matches the <code>CrossEntropyLoss</code> used in training.	DRIVER
<code>.save(dp)</code>	Writes <code>args.json</code> (<code>embedding_dim</code> , <code>num_classes</code> , <code>dropout_p</code>) and <code>model.pt</code> (the <code>state_dict</code>).	Produces the two files a checkpoint needs. Called as <code>model.module.save</code> under DDP.	WORKER

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>FinetunedLLM.load(args_fp, dict_fp)</code> classmethod	Builds the BERT architecture with <code>from_pretrained</code> , then overwrites it with your fine-tuned <code>state_dict</code> .	A <code>state_dict</code> holds tensors but no architecture, so the model has to be rebuilt first. Used after training.	DRIVER

Batching dataset rows → device tensors

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>get_device()</code> <code>@lru_cache</code>	Inside a worker: the device Ray assigned to that worker. In the notebook: <code>cuda</code> if available, else <code>cpu</code> . Looked up once per process.	Batches have to be on the same device as the model, and the same helper has to work in the notebook too.	BOTH
<code>pad_array(arr, dtype=np.int32)</code>	Zero-pads variable-length rows to the longest row in this batch.	Rows come from different preprocessing batches with different padded lengths, and a tensor needs one length.	BOTH
<code>collate_fn(batch)</code>	Pads ids and masks, converts to tensors (ids/masks <code>int32</code> , targets <code>int64</code>) on <code>get_device()</code> .	<code>iter_torch_batches</code> calls it once per batch. <code>CrossEntropyLoss</code> needs <code>int64</code> targets.	BOTH

Training

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>train_step(ds, batch_size, model, num_classes, loss_fn, optimizer)</code>	<code>model.train()</code> , then for each batch: <code>zero_grad</code> → forward → loss → backward → step. Returns the running-mean loss.	One epoch of learning on this worker's shard.	WORKER
<code>eval_step(ds, batch_size, model, num_classes, loss_fn)</code>	<code>model.eval()</code> + <code>inference_mode</code> : forward and loss only. Returns loss, <code>y_trues</code> , <code>y_preds</code> (argmax).	Measures generalization and feeds <code>val_loss</code> to the scheduler and the checkpoint ranking.	WORKER

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>train_loop_per_worker(config)</code>	Reads hyperparameters → seeds → shards → model + <code>prepare_model</code> → loss/optimizer/scheduler → epoch loop → save + <code>train.report</code> .	The single function Ray copies into every worker. It's the whole training job from one worker's point of view.	WORKER

Config & launch

NAME	WHAT IT DOES	WHY IT'S NEEDED	RUNS ON
<code>train_loop_config</code>	Dict of hyperparameters. Arrives in each worker as <code>config</code> .	One function, many configs. Ray Tune can sweep it later.	DRIVER
<code>ScalingConfig</code>	Number of workers, GPU or not, CPU/GPU per worker.	Tells Ray how many copies of the loop to start and what each one gets.	DRIVER
<code>RunConfig</code> + <code>CheckpointConfig</code>	Run name, <code>storage_path</code> ; keep the best N checkpoints by <code>val_loss</code> .	Where checkpoints go and which ones survive. Leave out <code>num_to_keep</code> to keep every epoch's.	DRIVER
<code>DataConfig</code>	<code>datasets_to_split=["train", "val"]</code> , <code>preserve_order=True</code> .	Shard both datasets across workers, and deliver blocks in a fixed order for reproducibility.	DRIVER
<code>del llm, model;</code> <code>gc.collect()</code>	Drops the notebook's exploration copies of SciBERT and frees them now.	Workers load their own copies, and on one machine they share the same RAM.	DRIVER
<code>TorchTrainer(...)</code> → <code>.fit()</code>	Builds the trainer from everything above, then runs it and returns <code>results</code> .	The entry point that launches distributed training.	DRIVER

Distributed training, mapped to your code

The general recipe for data-parallel training on the left, and the line in your notebook that does it on the right.

PREPARE, ONCE

01	Start a cluster	<code>ray.init()</code>	DRIVER
02	Load data as a distributed dataset (blocks)	<code>load_data()</code>	DRIVER
03	Split train / validation	<code>stratify_split()</code>	DRIVER
04	Fit preprocessing on train, apply to both	<code>CustomPreprocessor.fit_transform</code> / <code>.transform</code>	DRIVER
05	Cache the processed data	<code>.materialize()</code>	DRIVER
06	Describe the run: hyperparameters, workers, checkpoints, sharding	<code>train_loop_config</code> , <code>ScalingConfig</code> , <code>RunConfig</code> , <code>DataConfig</code>	DRIVER
07	Launch N worker processes	<code>TorchTrainer(...).fit()</code>	DRIVER

INSIDE EACH WORKER, ONCE

08	Seed, then take this worker's shard	<code>set_seeds()</code> , <code>train.get_dataset_shard()</code>	WORKER
09	Build the model, move it to the device, wrap it for gradient sync	<code>FinetunedLLM</code> , <code>prepare_model</code> (uses the worker's device + DDP)	WORKER
10	Split the global batch across workers	<code>batch_size // get_world_size()</code>	WORKER

EVERY BATCH

11	Load a batch onto the device	<code>iter_torch_batches</code> → <code>collate_fn</code> → <code>pad_array</code> , <code>get_device()</code>	WORKER
12	Forward, loss, backward; gradients averaged across workers; identical update	<code>train_step()</code> (DDP all-reduce runs inside <code>backward()</code>)	WORKER

EVERY EPOCH

13	Validate and adjust the learning rate	<code>eval_step()</code> , <code>scheduler.step(val_loss)</code>	WORKER
14	Checkpoint and report metrics (workers sync here)	<code>model.module.save</code> , <code>Checkpoint.from_directory</code> , <code>train.report</code>	WORKER

AFTER TRAINING

15	Collect results and pick the best checkpoint	<code>results.metrics_dataframe</code> , <code>results.best_checkpoints</code>	DRIVER
16	Load it for inference	<code>FinetunedLLM.load</code> → <code>predict</code> / <code>predict_proba</code> → <code>decode</code>	DRIVER

Current settings & the vocabulary

train_loop_config

dropout_p	0.5
lr	1e-4
lr_factor	0.8 · multiply lr on plateau
lr_patience	3 epochs without improvement
num_epochs	5
batch_size	32 total → 16 per worker
num_classes	len(tags)

Run

workers	2 × {CPU: 4, GPU: 0}
checkpoints	keep 1, best by val_loss (min)
storage	~/ray_results/llm
data	764 rows → 610 train / 154 val

Terms

block	Ray Data's unit of storage and parallel work. Set by <code>override_num_blocks</code> .
shard	The set of blocks one worker trains on. One per worker, no overlap.
batch	Rows sliced from a shard for one step. Can span blocks; never spans shards.
worker	A process running <code>train_loop_per_worker</code> with its own model copy.
device	The hardware a worker computes on: cpu or one cuda:k.
parameters()	Trainable tensors. <code>named_parameters()</code> adds names.
state_dict()	Named parameters + buffers. What gets saved and loaded.
DDP	Wrapper that averages gradients across workers. Your model sits at <code>model.module</code> .

Left out as exploration: EDA and word clouds, the pandas `train_test_split` check, the Claude zero-/few-shot baselines and their plots, the `llm` inspection cells, and the reference-only `Preprocessor` class.